

Small Matrices - Requirements and Restrictions

Dissertation reproducibility materials

This document records the software requirements, benchmark definition, computational assumptions and reproducibility limitations for the Small Matrices baseline. It is intended to prevent accidental changes to the experiment when the repository material is reused or reproduced.

1. Required software

The benchmark is designed to run in a standard local Python environment. Anaconda or Miniconda is suitable. The required Python packages are:

- **Python 3** - the programming language used to execute the scripts.
- **NumPy** - numerical arrays and pseudorandom-number generation.
- **SciPy** - sparse matrices, LinearOperator objects and the four solver implementations.
- **pandas** - tabulation and CSV output.
- **matplotlib** - generation of the benchmark figures.

2. Suggested Conda environment

A clean environment can be created with the following commands. The exact Python version is not itself part of the mathematical experiment, but recording the actual Python and package versions is recommended for archival reproduction.

```
conda create -n small-matrices python=3.11 numpy scipy pandas matplotlib
conda activate small-matrices
```

3. Benchmark restrictions

The following settings define the controlled Small Matrices repository baseline. They should not be changed silently if the resulting run is described as a reproduction of the dissertation baseline.

- **Scale:** Small Matrices baseline; this is not the large-scale matrix-free experiment.
- **Columns:** $n = 50$.
- **Rows:** $m = 400, 500, 1000, 1100$ and 2100 .
- **Solvers:** Direct, CG, LSQR and LSMR.
- **Patterns:** random, banded and block.
- **Spectra:** clustered, exponential, linear and polynomial.
- **Repeats:** 10 per configuration.
- **Noise:** relative noise level 0.01.
- **Stopping controls:** tolerance $1e-8$ and maximum 5000 iterations.
- **Random seed:** 42.

4. Condition parameter versus realised condition number

The values 10, 100, 1000 and 10000 are used as **condition parameters** when constructing the matrix spectrum. They should not automatically be interpreted as the exact numerical condition number of the final matrix.

After sparsity and the selected matrix structure are imposed, the realised numerical conditioning can differ from the requested parameter. The repository documentation therefore uses the term condition parameter for these four benchmark inputs.

5. Regularisation and solver formulation

The unregularised case uses $\lambda = 0$. Positive λ values introduce the ridge/Tikhonov-style penalty.

CG operates on the regularised normal-equation operator. LSQR and LSMR use damping. The Direct method explicitly solves the corresponding normal-equation system.

6. Reference benchmark configuration

The table below is the compact reference definition of the controlled repository run.

Parameter	Configuration
m	400, 500, 1000, 1100, 2100
n	50
Sparsity	0, 0.25, 0.5, 0.75, 0.9
Condition parameter	10, 100, 1000, 10000
Lambda	0, 1e-8, 1e-7, 1e-6, 1e-5, 1e-2, 1
Patterns	random, banded, block
Spectra	clustered, exponential, linear, polynomial
Noise / tolerance / max iterations	0.01 / 1e-8 / 5000
Repeats / seed	10 / 42

7. Generated CSV and PNG files

CSV files produced by the benchmark are generated data products, not source dependencies. PNG files produced by the plotting script are generated analysis products, not source dependencies.

After confirming that the scripts regenerate the required outputs, the CSV and PNG files may be removed from the repository. Removing the generated output directory does not change the benchmark definition or the source code.

For a clean reproducibility repository, the preferred contents are therefore the source Python script, the plotting script and the explanatory documentation, rather than previously generated result files.

8. Hardware and runtime limitations

- Runtime is machine-dependent and should not be expected to match the dissertation exactly on another computer.
- CPU architecture, BLAS/LAPACK backend, SciPy/NumPy versions and system load can affect timings.
- For fair solver comparisons, all four solvers should be run in the same environment and under comparable conditions.
- The benchmark is intended for small matrices and should not be treated as a substitute for the dissertation's large-scale experiments.

9. Reproducibility limitations

A fixed seed provides deterministic pseudorandom inputs for the recorded implementation, but exact bit-for-bit and timing reproduction is not guaranteed across arbitrary software environments.

Reproducibility should therefore be understood primarily as the ability to regenerate the same experimental procedure and obtain numerically consistent behaviour, rather than as a guarantee of identical execution times or identical floating-point output on every computer.

10. Repository hygiene

- Do not silently change benchmark constants and continue to describe the run as the dissertation baseline.
- If a modified experiment is created, document the modification separately.
- Keep source code and explanatory documentation under version control.

- Keep generated results and figures out of the repository unless archival storage is explicitly required.

Important: If the source code is later revised, this requirements document must be revised at the same time so that the repository documentation and executable benchmark remain consistent.